YOLOv8 vs YOLO11 for Tomato Segmentation: Efficiency Analysis for Agricultural Robotics

Henrique Moreira Amorim
Federal University of Uberlandia
Uberlândia, Brazil
henrique.amorim@ufu.br

Lirong Xiang
Cornell University
New York, United States
lxiang@cornell.edu

Jefferson R. Souza
Federal University of Uberlandia
Uberlândia, Brazil
jrsouza@ufu.br

***Abstract*—Autonomous agricultural robots require instance segmentation models that balance accuracy and computational efficiency for deployment on resource-constrained embedded platforms. Although recent YOLO architectures offer native support for instance segmentation, their behavior under architectural modifications remains insufficiently characterized for robotic agricultural applications. This work presents a systematic evaluation of twelve YOLO-based instance segmentation architectures for on-plant tomato segmentation, using the TomatoTest synthetic dataset (21,384 images). The evaluated models include YOLOv8-Seg, YOLO11-Seg, an attention-enhanced variant based on Squeeze-and-Excitation (SE) modules, and five structurally simplified architectures, all trained on the same data splits and evaluated under a common protocol.**

Results show that YOLO11L-Seg achieves performance equivalent to YOLOv8L-Seg within ± 1 percentage point (F1: 87.85% vs. 88.25%, Mask mAP@50: 96.22 vs. 96.63) while requiring 40% fewer parameters, and this equivalence holds at all three scales. None of the models showed overfitting, with validation–test gaps below 0.6 percentage points. Controlled experiments that retrained YOLOv8L-Seg and YOLOv8L-Slim under matching recipes showed that the training recipe changed F1 by about 0.9 percentage points, more than any architectural modification: under the same recipe, every attention-based or structurally reduced variant was equivalent to YOLOv8L-Seg within ± 1 pp. We also show that test-time augmentation with vertical flips lowered the F1-score of models trained without vertical flips to 69–76%. These results describe the TomatoTest synthetic distribution and indicate that, in this scenario, segmentation accuracy is determined more by the training recipe than by the architectural modifications evaluated.

***Index Terms*—Instance segmentation, YOLO, Deep learning, Tomato detection, Agricultural robotics.**

I. Introduction

Tomato is one of the most important horticultural crops worldwide, with global production exceeding 180 million tons annually [1], making it the most widespread and cultivated vegetable crop [2]. Computer vision methods for automatic fruit identification on tomato plants enable automated counting, supporting the evaluation of different planting, fertilization, and management techniques.

Deep learning methods based on Convolutional Neural Networks (CNNs) have become the dominant approach for fruit detection, counting, and estimation, demonstrating robust performance even in scenarios with high visual complexity [3], [4].

The YOLO (You Only Look Once) architecture is a reference for real-time object detection, and recent versions such as YOLOv8-Seg and YOLO11-Seg extend its native support to instance segmentation, which plays a fundamental role in agricultural applications by enabling individual fruit separation under partial occlusions and high visual variability.

Despite the growing adoption of YOLO-based models in agricultural computer vision, existing studies primarily compare different model versions or scales; the impact of architectural modifications—attention mechanisms and structural reduction—on segmentation accuracy, efficiency, and generalization remains insufficiently understood under controlled conditions, which is particularly relevant for resource-constrained deployments such as autonomous agricultural robots [5].

Therefore, this work investigates the impact of architectural design choices on tomato instance segmentation through a systematic evaluation of YOLOv8-Seg and YOLO11-Seg models across multiple scales, as well as attention-based and efficiency-oriented architectural modifications under a controlled experimental setup on a synthetic benchmark.

The main contributions of the study are:

- A systematic evaluation of twelve YOLO-based instance segmentation architectures, including YOLOv8-Seg, YOLO11-Seg, attention-enhanced models, and structurally simplified variants.
- A consistent evaluation protocol, with validation-selected confidence thresholds and paired statistical tests, showing that none of the evaluated models overfits and that test-time augmentation must be consistent with the augmentations used in training.
- A comparative assessment showing that YOLO11 matches YOLOv8 within ± 1 pp with up to 40% fewer parameters, and that this equivalence holds at all three scales.
- A controlled experiment that retrains YOLOv8L-Seg and YOLOv8L-Slim under matching recipes to separate the effect of the training recipe from that of the architectural modifications, showing that the recipe affects F1 more than any modification evaluated and that YOLOv8L-Seg's parameter count can be reduced by up to $4\times$ without a loss of accuracy.

II. Related work

A. Deep learning for fruit detection

CNNs have been widely employed in fruit detection, counting, and segmentation tasks across crops such as apples, citrus fruits, mangoes, and tomatoes, demonstrating robust performance under varying agricultural conditions, including occlusions, illumination changes, and fruit clustering [3], [6]. More recently, instance segmentation approaches have gained attention because they enable the identification of individual fruits at the pixel level, supporting applications such as yield estimation, growth monitoring, and robotic harvesting [7].

Zhang et al. [6] proposed a system for detecting and tracking citrus fruits in the field, evaluating multiple YOLO versions (YOLOv3, YOLOv4, and YOLOv5) and reporting strong detection accuracy under lighting variations and occlusions. However, such studies focus on cross-version benchmarking for bounding-box detection and do not examine how architectural modifications affect instance segmentation or model generalization.

Deng et al. [8] proposed SE-YOLO, a Sobel-enhanced YOLO variant for lightweight real-time tomato detection targeting edge deployment, showing that task-specific architectural modifications can improve efficiency without substantial accuracy loss.

B. Evolution of YOLO Architectures

The YOLO architecture has undergone several revisions to improve the balance between accuracy and computational efficiency; recent versions, such as YOLOv8 and YOLO11, incorporate native support for instance segmentation and refinements that improve feature extraction while reducing computational cost [9].

According to the official documentation, the YOLO11m model achieves higher average precision (mAP) on the COCO dataset than YOLOv8m while using 22% fewer parameters. Consequently, evaluating how such modifications affect accuracy, generalization, and computational efficiency becomes particularly relevant for agricultural computer vision applications.

C. Synthetic Data for Agriculture

Synthetic data has emerged as an alternative for addressing the scarcity of annotated agricultural datasets, particularly in tasks that require pixel-level labeling and large-scale data collection. For example, He et al. [7] generated a synthetic dataset of tomato plants using the Unreal Engine 5 game engine and employed a semantic segmentation algorithm with domain adaptation to quantify diseases in the field.

D. Architectural modifications

. Architectural modifications have been widely investigated to improve the balance between task performance and computational efficiency, particularly for resource-constrained platforms such as mobile and embedded vision systems [10].

Recent studies have explored different strategies to reduce computational complexity while preserving accuracy. Network pruning techniques remove redundant parameters and channels from deep architectures [11], whereas compound scaling approaches adjust network width, depth, and resolution in a coordinated manner to improve efficiency [12].

Attention mechanisms have also been incorporated into convolutional architectures to improve feature representation. Among them, Squeeze-and-Excitation (SE) blocks explicitly model channel-wise dependencies and have demonstrated effectiveness in several visual recognition tasks [13].

Instance segmentation architectures such as Mask R-CNN [14] perform well on general-purpose benchmarks, but their computational overhead and two-stage pipeline limit real-time agricultural use, motivating YOLO-based single-stage approaches for resource-constrained deployments. The impact of architectural modifications on agricultural instance segmentation, however, remains insufficiently investigated.

III. Proposed methodology

A. Dataset

This work was developed using the TomatoTest dataset [7], a synthetic dataset composed of 21,384 pairs of tomato plant images generated with the Unreal Engine 5 graphics engine. Each sample consists of RGB images from the left and right cameras, corresponding depth maps, semantic segmentation labels, and instance masks associated with the left image.

The three-dimensional plants were obtained from SketchFab, and foliar disease textures from the PlantVillage dataset [15]; synthetic data generates varied visual scenarios in a controlled manner, reducing manual annotation effort and enabling large-scale training sets [16].

He et al. [7] validated this synthetic dataset using Unsupervised Domain Adaptation (UDA), showing that models trained on TomatoTest combined with UDA achieved competitive performance on real field images, supporting its use for comparative studies in agricultural vision tasks. In this work, the dataset serves as a controlled benchmark for comparative architecture evaluation.

TABLE I
Characteristics of the TomatoTest dataset

Feature	Value
Total number of samples	21,384 pairs
Original resolution	2448 × 2048 px
Modalities	RGB and depth
Available labels	Semantic segmentation and instance
License	CC BY-NC 4.0

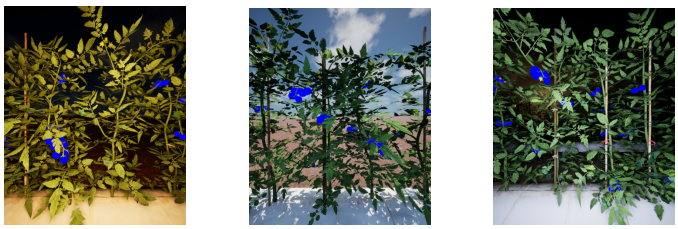


Fig. 1. Examples of the TomatoTest synthetic dataset [7] with instance segmentation masks (blue) showing individual tomatoes under varying conditions.

B. Data Preprocessing and Preparation

Regions smaller than 800 pixels were first removed from the semantic masks, and the images were divided into training, validation, and test sets (70/20/10%) to prevent data leakage. Each set was then divided into 640×640-pixel patches using a *sliding-window* strategy with 50% overlap, keeping only patches in which tomatoes cover at least 3% of the area. Instance labels were obtained by connected-component analysis and stored in the YOLO segmentation format; the *watershed* algorithm was used only to separate touching fruits when counting.

C. Model Architecture

Twelve model configurations were evaluated, grouped into three categories: (1) base models of the YOLOv8-Seg and YOLO11-Seg architectures; (2) an attention-based variant incorporating Squeeze-and-Excitation (SE) modules [13]; and (3) five structural-reduction variants.

The *Base* models include the YOLOv8-Seg and YOLO11-Seg architectures, each evaluated at three scales (*Small*, *Medium*, and *Large*) without structural modifications. YOLO11-Seg represents the most recent generation of YOLO segmentation models from Ultralytics, incorporating architectural refinements designed to improve parameter efficiency while maintaining native support for instance segmentation.

The *Attention (SE)* variant incorporates Squeeze-and-Excitation modules [13] (Fig. 2) in the multi-scale fusion layers (P3, P4, P5) of the Feature Pyramid Network (FPN) neck, adding channel recalibration without altering the topology (46.0M parameters, 0.16% overhead).

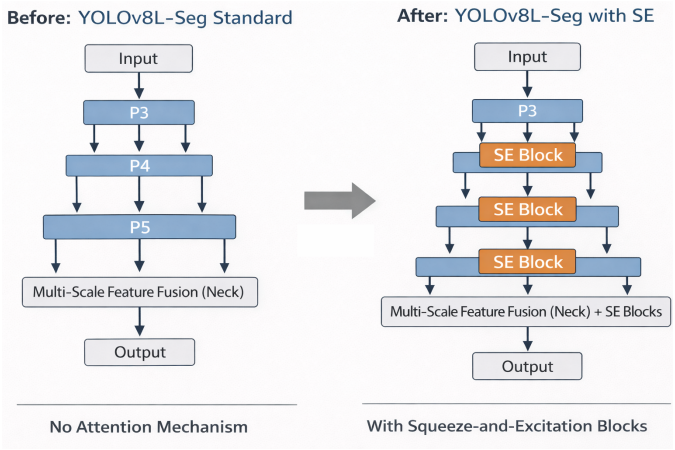


Fig. 2. Comparison between the standard YOLOv8L-Seg architecture (left) and the proposed YOLOv8L-SE variant (right). SE blocks (orange) are inserted at FPN levels P3, P4, and P5 for channel-wise feature recalibration.

Finally, variants grouped as *Structural Reduction* modify the width and/or depth of the YOLOv8L-Seg architecture by reducing C2f channels and blocks at different pyramid levels [11], [12], [17], to investigate how reduced network capacity affects accuracy, efficiency, and generalization.

The four variants apply different combinations of width reduction and C2f block pruning to the YOLOv8L-Seg backbone (9.8–11.8M parameters): *YOLOv8L-Slim* halves the channel width (0.5); *YOLOv8L-SlimDepth* additionally reduces C2f blocks at P2–P5 (3/6/6/3 → 1/3/3/1); *YOLOv8L-SlimEdge* reduces only P2 (3 → 1); and *YOLOv8L-SlimSelective* applies selective pruning across pyramid levels (1/2/4/2).

The *EdgeOnly* variant was designed as an ablation study, restricting simplifications to level P2 of the feature pyramid ($C2f[3 \rightarrow 1]$) to investigate the isolated impact of reduction on high spatial resolution features, keeping the other levels and the original width unchanged (45.8M).

D. Training settings

TABLE II
TRAINING CONFIGURATIONS. CONFIG. A: SIX BASE MODELS; CONFIG. B: SE, SLIM AND EDGEONLY VARIANTS.

Parameter	Config. A	Config. B
Framework	Ultralytics 8.3.206 [9]	
Epochs / early stopping	100 / patience of 20 epochs	
Batch size / input size	6 / 640 × 640	
Optimizer	AdamW, lr 0.001, momentum 0.937	
Mosaic augmentation	Disabled for the last 10 epochs	
Training patches	21,465 (random 30% subset[a]: 6,439)	
Random seed	0	42
Mixed precision (AMP)	Enabled	Disabled
Warm-up epochs	5	3
Rotation / mixup / copy-paste	15° / 0.15 / 0.1	0 / 0 / 0
Vertical flip probability	0.1	0
Hardware	NVIDIA RTX 3060 (12 GB)	

[a] YOLOv8L-SE and a YOLOv8L-Seg control (Section IV-D).

Two training configurations were used (Table II): Config. A for the six base models and Config. B for the modified variants; YOLOv8L-SE used a random 30% subset of the training patches. Each YOLOv8/YOLO11 pair shares the same data and configuration. To isolate the effect of the training recipe, YOLOv8L-Seg and YOLOv8L-Slim were retrained under the other configuration (YOLOv8L-Seg also under Config. B with SE's 30% subset; Section IV-D), and YOLOv8M-Seg/YOLO11M-Seg were retrained under Config. A with the full training set (Table IV).

E. Evaluation metrics

The F1-score was adopted as the primary metric, as it balances precision and recall and provides an intuitive indicator of detection performance under the occlusion variability and class imbalance typical of agricultural scenes. Each full-resolution test image is processed with the same 640×640 sliding window (50% overlap, including a final window aligned with the image border), without test-time augmentation (TTA). The instance masks of each window are merged, overlapping windows are averaged, and F1 and IoU are computed pixel-wise against the ground-truth mask and averaged over images. The confidence threshold of each model is the one that maximizes F1 on 1,000 randomly selected validation images. The 52 test images without fruit are excluded from these averages (2,084 of the 2,136 test images remain), since F1 is undefined when both masks are empty. Differences between models are tested with the paired Wilcoxon signed-rank test, 95% bootstrap

confidence intervals resampled by scene (images captured at the same time are not independent), and a two one-sided test (TOST) with a ± 1 percentage point (pp) equivalence margin. Mask-based metrics — mask mAP@50 and mask mAP@50:95 — are additionally reported following the COCO evaluation protocol on the 2,988 test patches (640×640, fruit area $\geq$3%).

IV. Results

A. Overall performance comparison

The results indicate that YOLO11 models achieve segmentation performance comparable to their YOLOv8 counterparts while requiring substantially fewer parameters. In particular, YOLO11L-Seg achieved an F1-score of 87.85, remaining within 0.40 pp of YOLOv8L-Seg while reducing the parameter count from 46.0M to 27.6M parameters (approximately 40%).

TABLE III
PERFORMANCE COMPARISON BETWEEN THE EVALUATED ARCHITECTURES (TEST SET; SECTION III-E). TOP: BASE MODELS (CONFIG. A); BOTTOM: MODIFIED VARIANTS (CONFIG. B).

Model	Params (M)	F1	IoU	mAP@50	mAP@50:95
YOLOv8L-Seg	46.0	88.25	79.81	96.63	**86.85**
YOLOv8S-Seg	11.8	88.08	79.49	96.01	84.35
YOLO11L-Seg	27.6	87.85	79.21	96.22	85.93
YOLO11M-Seg	22.4	88.47	80.08	96.52	85.81
YOLO11S-Seg	10.1	87.48	78.66	95.44	83.49
YOLOv8M-Seg	27.3	87.83	79.16	96.46	86.08
YOLOv8L-EdgeOnly	45.8	**89.53**	**81.77**	**97.22**	86.48
YOLOv8L-Slim	11.8	89.28	81.38	96.70	84.53
YOLOv8L-SlimDepth	9.8	89.17	81.20	96.65	83.97
YOLOv8L-SlimEdge	11.7	88.89	80.90	96.63	84.37
YOLOv8L-SlimSelective	10.4	88.88	80.80	96.70	84.21
YOLOv8L-SE†	46.0	88.68	80.54	96.10	84.45

†Trained on a random 30% subset (Table II).

All twelve models reached F1-scores between 87.48 and 89.53, a range of 2.05 pp. The modified variants (Config. B) obtained F1-scores between 88.68 and 89.53, about 1 pp above the Config. A models. Section IV-D isolates the training recipe from the architectural modifications and shows that this difference is caused by the recipe, not by the architecture.

The Slim variants (9.8–11.8M) have about the same size as YOLOv8S-Seg (11.8M) and reached F1-scores 0.8–1.2 pp higher (YOLOv8L-Slim: +1.20 pp, 95% CI [0.96, 1.45], $p < 0.001$); Section IV-D shows that this advantage is explained by the training recipe rather than by the architecture.

The attention-based YOLOv8L-SE variant did not change segmentation performance meaningfully (F1 88.68, equivalent to YOLOv8L-Seg within ± 1 pp, TOST), even though it was trained on 30% of the patches; the SE blocks added only a marginal overhead (0.16%) without a clear benefit.

The results also show that the EdgeOnly variant achieved the highest F1-score (89.53) and Mask mAP@50 (97.22), indicating that reducing the C2f blocks at P2 did not harm segmentation at standard IoU thresholds. At stricter thresholds, however, YOLOv8L-Seg (Mask mAP@50:95 = 86.85) outperforms EdgeOnly (86.48), suggesting a small loss in fine-grained mask quality.

Fig. 3 illustrates the model's ability to separate adjacent fruits under varying occlusion conditions.



Fig. 3. Qualitative example of tomato instance segmentation using the YOLOv8L-Seg model on a test image. Predicted instance masks are overlaid on the original image, with per-instance confidence scores.

B. Comparison between YOLOv8 and YOLO11

TABLE IV
COMPARISON BETWEEN YOLOV8 AND YOLO11 F1-SCORES ON DIFFERENT SCALES USING TEST IMAGES. ΔF1 = YOLO11 − YOLOv8, WITH THE 95% SCENE-LEVEL BOOTSTRAP CI. ALL DIFFERENCES ARE SIGNIFICANT (WILCOXON, $p < 0.001$) AND EQUIVALENT WITHIN ± 1 PP (TOST).

Scale	F1 v8	F1 v11	ΔF1 [95% CI]	Params v8	Params v11	ΔParams
Large	88.25	87.85	−0.40 [−0.61, −0.18]	46.0M	27.6M	-40.0%
Medium	87.83	88.47	+0.64 [+0.47, +0.82]	27.3M	22.4M	-17.9%
Small	88.08	87.48	−0.59 [−0.85, −0.34]	11.8M	10.1M	-14.4%

YOLO11 models consistently maintained performance comparable to YOLOv8 across all evaluated scales while requiring substantially fewer parameters — YOLO11L-Seg, for instance, remained within 0.40 pp of YOLOv8L-Seg while reducing the parameter count by 40% (27.6M vs. 46.0M), making it a promising alternative for resource-constrained agricultural vision applications [5]. With more than 2,000 paired test images, differences of 0.4–0.6 pp are statistically detectable ($p < 0.001$), but the confidence intervals remain inside the ± 1 pp equivalence margin at all scales, so the two versions are equivalent for practical purposes.

Interestingly, the medium-scale configuration achieved a slight improvement in F1-score (+0.64) while reducing the parameter count by 17.9%, suggesting that architectural refinements introduced in YOLO11 can improve parameter efficiency without sacrificing segmentation performance.

C. Generalization and evaluation protocol

Table V compares validation and test F1 computed with the same pipeline: the gap was between −0.51 and +0.05 pp for all twelve models, showing no sign of overfitting, as expected since both sets come from the same synthetic distribution.

The evaluation protocol, however, had a strong effect on the modified variants. We also evaluated a four-view TTA

TABLE V
GENERALIZATION ANALYSIS. F1 ON 1,000 VALIDATION IMAGES AND ON THE TEST SET, COMPUTED WITH THE SAME PIPELINE AND THE CONFIDENCE THRESHOLD (CONF) SELECTED ON VALIDATION. GAP = F1 (VAL) − F1 (TEST). †TRAINED ON A RANDOM 30% SUBSET.

Model	conf	F1 (Val)	F1 (Test)	Gap (pp)
YOLOv8L-Seg	0.45	87.96	88.25	−0.28
YOLOv8M-Seg	0.40	87.46	87.83	−0.37
YOLOv8S-Seg	0.40	87.69	88.08	−0.38
YOLO11L-Seg	0.35	87.68	87.85	−0.17
YOLO11M-Seg	0.40	88.06	88.47	−0.41
YOLO11S-Seg	0.45	87.18	87.48	−0.30
YOLOv8L-SE†	0.40	88.72	88.68	+0.05
YOLOv8L-Slim	0.35	89.04	89.28	−0.23
YOLOv8L-SlimDepth	0.40	88.71	89.17	−0.46
YOLOv8L-SlimEdge	0.50	88.50	88.89	−0.39
YOLOv8L-SlimSelective	0.40	88.37	88.88	−0.51
YOLOv8L-EdgeOnly	0.40	89.39	89.53	−0.14

(original, horizontal flip, vertical flip, both), keeping a pixel only if more than half of the views detect it. With a fixed confidence threshold of 0.20 and no border window, the Config. B variants (trained without vertical flips) reached F1-scores of only 68.8–75.6, versus 82.6–84.0 for the Config. A models (trained with vertical flips): the flipped views miss most fruits, so pixels seen only by the two upright views fall exactly on the 0.5 threshold and are discarded. TTA must therefore be consistent with the augmentations used during training.

D. Architecture vs. training recipe

Table VI reports the effect of retraining YOLOv8L-Seg and YOLOv8L-Slim under the other configuration.

TABLE VI
EFFECT OF THE TRAINING RECIPE ON F1 (TEST SET). Δ = CONFIG. B − CONFIG. A.

Model	Config. A	Config. B	Δ [95% CI]
YOLOv8L-Seg	88.25	89.14	+0.90 [+0.62, +1.15]
YOLOv8L-Slim	88.49	89.28	+0.78 [+0.57, +1.00]

Changing only the training recipe raised F1 by 0.90 pp for YOLOv8L-Seg and 0.78 pp for YOLOv8L-Slim (Wilcoxon, $p < 0.001$). Under Config. B, every variant is equivalent to a YOLOv8L-Seg control trained under the same recipe within ±1 pp (TOST): Slim +0.14, SlimDepth +0.03, SlimEdge −0.25, SlimSelective −0.26, EdgeOnly +0.39, and SE (30% subset) +0.20 pp. Under Config. A, Slim is also equivalent to YOLOv8L-Seg (+0.25 pp). These results indicate that the training recipe affects accuracy more than the architectural modifications evaluated, and that YOLOv8L-Seg's parameter count can be reduced by up to 4× without a loss of accuracy.

E. Counting accuracy

Predicted and ground-truth instance counts for YOLOv8L-Seg across the 2,136 test images are strongly correlated (Pearson r = 0.957, $R^2 = 0.915$, $p < 0.001$; $y = 1.00x + 0.64$), confirming alignment across varying fruit densities (Fig. 4).

The distribution of absolute count errors further characterizes this alignment: 26.8% of test images yielded exact fruit counts, 84.5% showed errors of at most two fruits, and

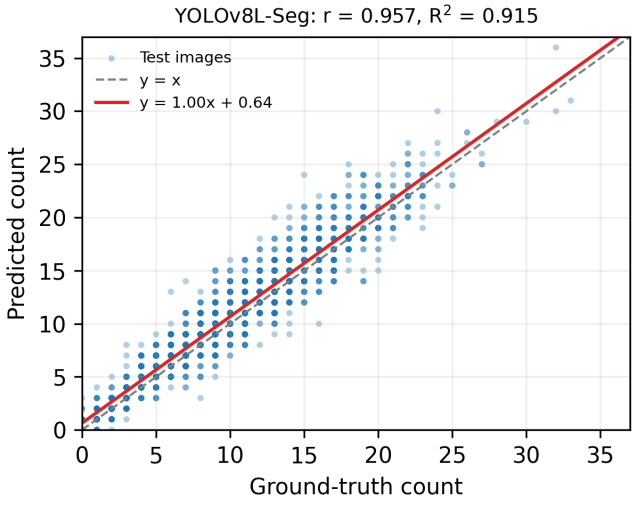


Fig. 4. Pearson correlation between ground-truth and predicted fruit counts for YOLOv8L-Seg across the 2,136 test images ($r = 0.957$, $R^2 = 0.915$, $p < 0.001$). The regression line $y = 1.00x + 0.64$ indicates strong alignment across varying fruit densities.

the mean absolute error reached 1.34 fruits per image. The modified variants obtained similar or slightly lower counting errors (MAE 1.20–1.25), and all models reached $r \geq 0.951$. On the 52 test images without fruit, the models marked fruit pixels in 23–42% of the images.

F. Inference time analysis

TABLE VII
INFERENCE COST ON 6,300 TEST PATCHES (640×640) FROM 150 TEST IMAGES: BATCH 1, FP32, NO TTA, 10 WARM-UP ITERATIONS, NVIDIA RTX 3060 (ULTRALYTICS 8.4.7). TIMES INCLUDE PRE- AND POST-PROCESSING; FPS = 1000/MS; SPEEDUP RELATIVE TO YOLOV8L-SEG.

Model	GFLOPs	GPU mem. (MB)	ms	FPS	Speedup	F1
YOLOv8L-Seg	210.7	406	32.1	31.1	1.00×	88.25
YOLOv8M-Seg	104.7	321	22.5	44.4	1.43×	87.83
YOLOv8S-Seg	40.2	232	13.6	73.7	2.37×	88.08
YOLO11L-Seg	132.6	339	28.9	34.5	1.11×	87.85
YOLO11M-Seg	113.5	320	22.8	43.8	1.41×	88.47
YOLO11S-Seg	33.1	228	15.9	62.7	2.02×	87.48
YOLOv8L-SE	210.8	406	31.9	31.3	1.01×	88.68
YOLOv8L-EdgeOnly	202.3	404	30.0	33.3	1.07×	89.53
YOLOv8L-Slim	54.3	231	18.8	53.1	1.71×	89.28
YOLOv8L-SlimEdge	52.2	231	18.2	55.0	1.77×	88.89
YOLOv8L-SlimSelective	45.6	226	16.6	60.4	1.94×	88.88
YOLOv8L-SlimDepth	45.4	223	16.2	61.9	1.99×	89.17

Table VII shows that inference speed varied much more than segmentation accuracy: F1 changed by at most 2.4 pp across models, while latency varied by a factor of 2.4. The SlimDepth variant achieves an almost 2× speed increase (16.2 ms vs 32.1 ms, corresponding to 61.9 FPS vs 31.1 FPS) compared to the reference YOLOv8L-Seg, with an F1-score 0.92 pp higher (89.17 vs 88.25; 95% CI [0.66, 1.18]). YOLOv8S-Seg was the fastest model (13.6 ms, 73.7 FPS, 40.2 GFLOPs), faster than all Slim variants. Pre-/post-processing took 0.8/1.2 ms per patch; peak GPU memory ranged 223–406 MB; a full 2448×2048 image (42 patches) took 0.57–1.35 s.

Since accuracy differences were small, the choice of model in this scenario depends mainly on computational cost. Among the base models, YOLOv8S-Seg offered the best trade-off (88.08 F1 at 73.7 FPS), and YOLO11L-Seg matched YOLOv8L-Seg within ± 1 pp with 40% fewer parameters. The Slim variants reached F1-scores equivalent to YOLOv8L-Seg (Section IV-D) at 53–62 FPS, so reducing YOLOv8L-Seg's parameter count by up to $4\times$ also reduces latency without a loss of accuracy. These measurements were obtained on a desktop GPU; the models are candidates for embedded deployment, pending validation on the target hardware.

V. Conclusions

This study presented a systematic evaluation of YOLO-based architectures for tomato instance segmentation, including YOLOv8-Seg, YOLO11-Seg, attention-enhanced models, and structurally simplified variants, under a common evaluation protocol. The main findings show that:

- YOLO11L-Seg offers performance equivalent to YOLOv8L-Seg within ± 1 pp (F1: 87.85% vs. 88.25%) with 40% fewer parameters, and the equivalence holds at all three scales (counting reliability confirmed by r = 0.957, $p < 0.001$).
- Under a consistent evaluation protocol, none of the twelve models showed overfitting (validation–test gaps between -0.51 and +0.05 pp). The low F1-scores obtained with a four-view TTA were caused by vertical flips at test time in models trained without them.
- Controlled experiments that retrained YOLOv8L-Seg and YOLOv8L-Slim under matching recipes showed that the training recipe changed F1 by 0.78–0.90 pp (Section IV-D), more than any architectural modification: under the same recipe, every variant was equivalent to YOLOv8L-Seg within ± 1 pp, so the Slim variants reduce parameters by up to $4\times$ and increase speed by up to $1.99\times$ (61.9 FPS) without loss of accuracy; YOLOv8S-Seg was the fastest model overall (73.7 FPS).

The limitations include a single training run (one random seed) per model, the exclusive use of synthetic data, and latency measured on a desktop GPU rather than on embedded hardware. Five of the six modified variants (SE, SlimDepth, SlimEdge, SlimSelective, and EdgeOnly) were trained only under Config. B and are compared against a same-recipe control (Section IV-D), not the Config. A models in Table III. The results describe performance on the TomatoTest synthetic distribution and do not establish equivalent real-field performance. However, prior work has demonstrated the viability of domain-adaptive learning from synthetic tomato imagery for real-field deployment [7], supporting the use of synthetic benchmarks for comparative architecture evaluation. Future work aims to retrain the remaining variants under Config. A to confirm the recipe effect seen for YOLOv8L-Slim, repeat experiments with multiple seeds, explore domain adaptation to real-field conditions, and evaluate speed-accuracy cost-benefit on embedded platforms (e.g., drones and robots).

References

[1] C. Akotowanou, E. Adjou, A. Olubi, D. S. Kougblenou, E. Ahoussi, and D. Sohounhloué, "The tomato (solanum lycopersicum l.) in community development: An overview focused on nutritional properties, agronomic constraints, recent achievements and future prospective," *International Journal of Frontiers in Biology and Pharmacy Research*, vol. 3, pp. 008–016, 11 2022.

[2] Global Alliance for Improved Nutrition, "Tomatoes: The world's most popular vegetable," 2018, accessed: 2026-01-23. [Online]. Available: https://www.gainhealth.org/resources/reports-and-publications/tomatoes-worlds-most-popular-vegetable

[3] A. Koirala, K. B. Walsh, Z. Wang, and C. McCarthy, "Deep learning — method overview and review of use for fruit detection and yield estimation," *Computers and Electronics in Agriculture*, vol. 162, pp. 219–234, 2019.

[4] X. Zhao, L. Wang, Y. Zhang *et al.*, "A review of convolutional neural networks in computer vision," *Artificial Intelligence Review*, vol. 57, p. 99, 2024.

[5] C. Beldek, E. Sariyildiz, and G. Alici, "A deployment-oriented benchmarking of you look only once (yolo) models for orange detection and segmentation in agricultural robotics," *Agriculture*, vol. 15, no. 20, p. 2170, 2025.

[6] W. Zhang, J. Wang, Y. Liu, K. Chen, H. Li, Y. Duan, W. Wu, Y. Shi, and W. Guo, "Deep-learning-based in-field citrus fruit detection and tracking," *Horticulture Research*, vol. 9, p. uhac003, 02 2022. [Online]. Available: https://doi.org/10.1093/hr/uhac003

[7] W. He, X. Li, Z. Zhang, Y. Chen, J. Zhang, D. Panthee, I. Meadows, and L. Xiang, "High-throughput robotic phenotyping for quantifying tomato disease severity enabled by synthetic data and domain-adaptive semantic segmentation," *Journal of Field Robotics*, vol. 42, pp. 657–678, 12 2024.

[8] X. Deng, T. Huang, W. Wang, and W. Feng, "Se-yolo: A sobel-enhanced framework for high-accuracy, lightweight real-time tomato detection with edge deployment capability," *Computers and Electronics in Agriculture*, vol. 239, p. 110973, 2025. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S0168169925010798

[9] G. Jocher *et al.*, "Ultralytics yolov8," 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[10] A. G. Howard, M. Zhu, B. Chen, D. Kalenichenko, W. Wang, T. Weyand, M. Andreetto, and H. Adam, "Mobilenets: Efficient convolutional neural networks for mobile vision applications," 2017. [Online]. Available: https://arxiv.org/abs/1704.04861

[11] Z. Liu, J. Li, Z. Shen, G. Huang, S. Yan, and C. Zhang, "Learning efficient convolutional networks through network slimming," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[12] M. Tan and Q. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *International conference on machine learning*. PMLR, 2019, pp. 6105–6114.

[13] J. Hu, L. Shen, and G. Sun, "Squeeze-and-excitation networks," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 7132–7141.

[14] K. He, G. Gkioxari, P. Dollar, and R. Girshick, "Mask r-cnn," in *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, Oct 2017.

[15] D. P. Hughes and M. Salathe, "An open access repository of images on plant health to enable the development of mobile disease diagnostics," 2016. [Online]. Available: https://arxiv.org/abs/1511.08060

[16] J. Tremblay, A. Prakash, D. Acuna, M. Brophy, V. Jampani, C. Anil, T. To, E. Cameracci, S. Boochoon, and S. Birchfield, "Training deep networks with synthetic data: Bridging the reality gap by domain randomization," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) Workshops*, June 2018.

[17] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature pyramid networks for object detection," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 2117–2125.